

Deliveroo Autonomous Agent

Project Report — Autonomous Software Agents, A.A. 2025–2026

Matteo Ranzi*

Luca Hardonk†

September 2026

Repository: <https://github.com/matteoranzi/Autonomous-Software-Agent---Deliveroo.js>

1 Introduction

This report documents the design and implementation of a team of autonomous software agents for the Deliveroo.js game, in which agents move on a grid map, pick up parcels whose reward decays over time, and deliver them to dedicated delivery tiles. The project comprises:

- **Agent A**, a Belief–Desire–Intention [1] agent written in TypeScript, designed in the spirit of practical, modular BDI implementations [5, 6], whose distinguishing features are (i) an event-driven belief-revision layer built on pluggable change-detection strategies, (ii) intention selection performed by a Monte Carlo Tree Search (MCTS) over a dynamically built, logical (desire-level) state-space graph, (iii) a Mamdani-style fuzzy inference system used as the common reward signal that makes heterogeneous metrics (cost, reward, urgency) comparable, and (iv) a layered planning stack combining A* pathfinding with an *anytime* PDDL planner able to push crates out of the way (Sokoban-style), obtained by extending the open-source Planning-as-a-Service (PaaS) infrastructure.
- **Agent B**, an LLM-controlled agent that translates natural-language requests into structured, bounded plans executed through a constrained interpreter, and that can drive a second agent through a typed Model Context Protocol (MCP) interface.

The report follows the structure of the agent itself: beliefs (Section 3), desires (Section 4), intentions and MCTS (Section 5), fuzzy scoring (Section 6), planning and PDDL (Section 7), the LLM agent (Section 8), and validation (Section 9).

2 Architectural Design

The BDI agent is organised as a pipeline of loosely coupled components, each behind an interface so that alternative implementations can be swapped without touching the deliberation loop (Figure 1). Raw socket payloads from the Deliveroo.js server are first normalised by a thin *adapter* layer (`io/adapters/`) into internal types, decoupling the belief base from the wire format. The `Belief` object is the single source of truth; every downstream component (desire generation, intention selection, cost estimation, planning) reads from it and never from the network directly.

Hybrid deliberation triggering. The deliberation cycle (regenerate desires → deliberate → execute) is triggered by three sources rather than a fixed tick: (i) *event-driven*, whenever a change-detection strategy reports a desire-relevant belief change; (ii) *on completion or failure* of the current plan; and (iii) a low-frequency *backstop timer* that catches reprioritisation caused purely by continuous reward decay, which is not a discrete sensing event. A purely periodic loop would either waste cycles when nothing changed or overlap expensive planning calls; a purely event-driven design would miss decay-driven changes. A re-entrancy guard serialises cycles: a trigger arriving while a cycle runs is remembered and replayed once, never dropped and never run concurrently.

*Matricola: 209449, Email: matteo.ranzi@studenti.unitn.it

†Matricola: 267750, Email: luca.hardonk@studenti.unitn.it

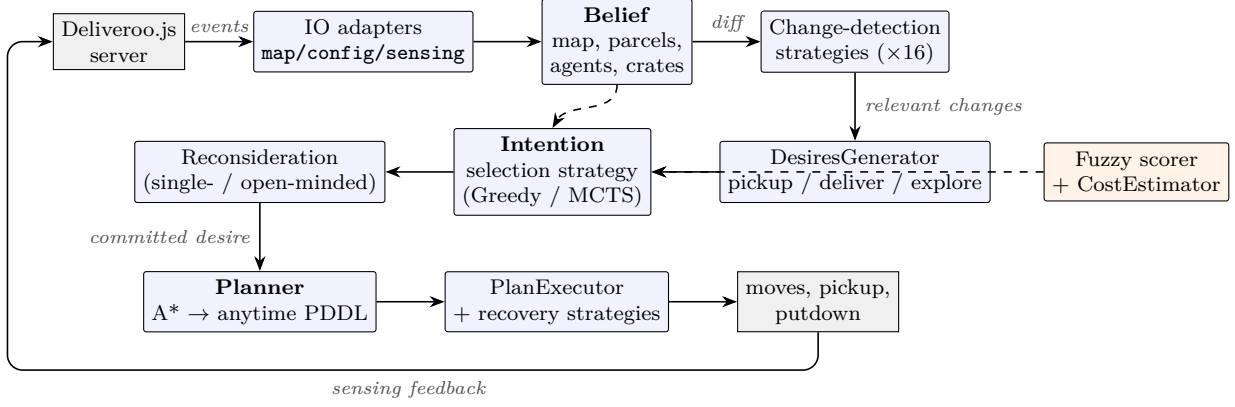


Figure 1: BDI agent architecture. Solid arrows show the main deliberation data flow; dashed arrows show read-only access to the belief base and the shared scoring services used by intention selection.

3 Beliefs

The belief base stores the static map (walkable tiles, spawner tiles, delivery tiles, crate-allowed tiles), the dynamic state of parcels, rival agents (with a bounded position history), crates, and the agent itself. Each tile additionally records a `lastTimeObserved` timestamp, which drives exploration (Section 4).

Consistent sensing composition. The server emits self-state (`onYou`) and environment sensing (`onSensing`) as separate, uncoordinated socket events. Applying them independently would let listeners observe a half-updated belief. The agent therefore buffers the latest `onYou` payload and applies it atomically together with the next `onSensing` update, so every belief listener always observes both halves of a tick at once.

Change-detection strategies. Belief revision is not limited to overwriting state: a pluggable set of sixteen *change-detection strategies* diffs each incoming update against the current belief and reports *what actually changed* at the semantic level, e.g. `NewParcelAppeared`, `FreeParcelVanished`, `RivalAgentPickedUpParcel`, `RivalAgentEnteredDeliveryTile`, `SelfAgentDeliveredParcel`, `CrateMoved`. Each triggered strategy also reports a *degree* quantifying how significant the change is. Only desire-relevant changes wake the deliberation cycle, so the expensive generate–score–plan pipeline runs when the world meaningfully changed, not on every sensing packet. Dedicated estimators infer facts the server does not state explicitly, such as which agent picked up a vanished parcel (`ParcelCarriedByEstimator`) or rival movement between tiles (`AgentTileTransitionEstimator`).

4 Desires

On every deliberation cycle `DesiresGenerator` regenerates the option set from the current belief and filters out invalid entries:

- **PickupParcelDesire** — one per known free parcel; the expected reward is the parcel’s current (decayed) reward, the cost the A*-estimated traversal cost.
- **DeliverParcelDesire** — one per delivery tile, applicable only while carrying; the expected reward is the sum of the rewards of all carried parcels, so that a multi-parcel delivery is correctly recognised as more valuable than a single-parcel one.
- **ObserveParcelSpawningTileDesire** — one per spawner tile currently outside the agent’s observation radius; its value is the tile’s *staleness* (a linear, capped function of the time since last observation), which steers the agent toward regions of the map it has not seen recently.

Each desire exposes a uniform evaluation record $\langle estimatedCost, expectedReward, urgency, risk \rangle$ and a cheap `isValid()` predicate encoding *hard* feasibility facts only (parcel still free and in place, delivery tile still reachable, spawner tile still unobserved). Soft, competitive judgements — “a rival is closer to that parcel” — are deliberately kept out of `isValid()` and left to the scoring layer, avoiding the oscillation caused by discarding desires on mere estimates. Validity is re-checked continuously: a desire invalidated mid-plan aborts the plan and triggers re-deliberation.

5 Intentions and Monte Carlo Tree Search

The `Intention` component commits to a single desire at a time. Both of its policies are injected behind interfaces and swappable at construction time:

- an `IIntentionStrategy` chooses *what* to commit to (`GreedyIntentionStrategy` or `MCTSIntentionStrategy`);
- an `IReconsiderationStrategy` decides *whether to keep* the current commitment as beliefs evolve mid-plan, implementing the canonical BDI commitment spectrum [1]: *single-minded* (keep while still valid), *open-minded* (drop a still-valid intention when a rival option scores strictly higher under the fuzzy scorer), and an intermediate policy that only switches to a higher-utility desire of the same kind.

5.1 Baseline: greedy selection

`GreedyIntentionStrategy` partitions desires into pickup/deliver/explore buckets and applies fixed priorities (deliver when at capacity, otherwise best pickup, explore as fallback), selecting the highest-utility desire within the chosen bucket. It is cheap and reactive but myopic: it cannot recognise that *pick up A, then B, then deliver both* dominates *deliver immediately after A*.

5.2 Logical state-space graph

To add lookahead, intention selection is cast as search over a *logical* state space whose granularity is the desire, not the individual movement action. A node abstracts the physical game state into the tuple

$$s = \langle position, carriedCount, visited \rangle,$$

where *position* is the goal tile of the last desire pursued, *carriedCount* the number of carried parcels, and *visited* the set of desires already consumed along the path from the root. An edge $s \xrightarrow{d} s'$ applies desire *d*: its traversal weight is the A* path cost between the two goal tiles (computed by a memoising `CostEstimator` from the *simulated* position, not the agent’s real one), and its immediate reward is the fuzzy desirability score of *d* evaluated in *s* (Section 6).

The graph is never materialised up front: a dynamic transition function (`availableActions`) generates the outgoing edges of a node on demand, enforcing the game’s logical constraints — *deliver* requires *carriedCount* > 0, *pickup* requires spare capacity, each desire is consumed at most once per path, and invalid desires are never surfaced. This lazy construction is what makes the search tractable: only the fraction of the graph actually explored by MCTS is ever built.

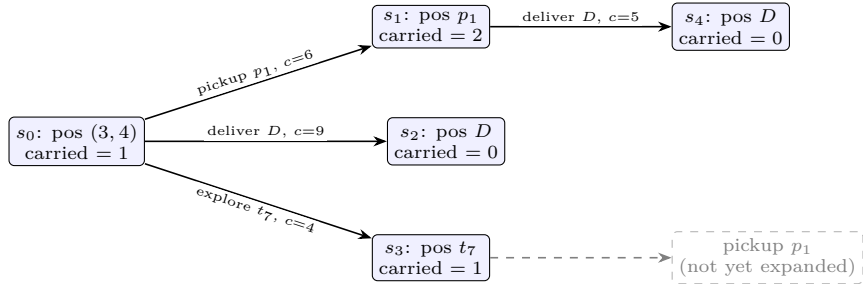


Figure 2: Fragment of the logical state-space graph. Nodes are desire-level abstractions of the game state; edges are desires weighted by memoised A* costs *c* and rewarded by the fuzzy scorer. The graph is expanded lazily, only where the search visits it.

5.3 MCTS over desire sequences

`MCTSIntentionStrategy` runs the four canonical MCTS phases [2] over this graph for a fixed iteration budget (Figure 3); casting the decision problem at the task level rather than the primitive-action level follows the way MCTS is applied to scheduling problems in dynamic, uncertain domains [3, 4]:

1. **Selection:** from the root (the agent’s real current state), descend through fully expanded nodes by maximising UCB1, $\bar{R}_i + c\sqrt{\ln N/n_i}$ with $c = \sqrt{2}$, balancing the exploitation of high-reward desire sequences against the exploration of rarely tried ones.

2. **Expansion:** materialise one untried desire edge of the reached node, computing the child state via the transition function of Section 5.2.
3. **Simulation:** perform a random rollout of desires from the child down to a fixed maximum depth (5 logical steps); the game has no terminal state, so the rollout value is the accumulated fuzzy reward along the sampled sequence.
4. **Backpropagation:** add the rollout value to the visit and reward statistics of every node on the path back to the root.

The core loop is deliberately close to the textbook formulation:

```
for (let i = 0; i < this.iterations; i++) {
  let node = root;
  // Selection: descend via UCB1 while fully expanded
  while (node.isFullyExpanded && node.children.length > 0
    && node.state.visited.size < this.maxDepth) {
    node = selectChild(node, this.explorationConstant);
  }
  // Expansion: materialise one untried desire edge
  if (!node.isFullyExpanded && node.state.visited.size < this.maxDepth) {
    node = await expand(node, desires, costEstimator, scorer);
  }
  // Simulation + Backpropagation
  const reward = await rollout(node.state, desires, costEstimator, scorer, this.maxDepth);
  backpropagate(node, reward);
}
```

After the budget is exhausted the strategy returns the *robust child* [2]: the root action with the highest visit count rather than the highest average reward, a standard choice that is less sensitive to a few lucky rollouts. Only the first desire of the best sequence is committed; the search is re-run on the next deliberation cycle against fresh beliefs, which keeps the lookahead honest in a dynamic, adversarial environment while the plan-level reactivity (Section 7) handles the execution details.

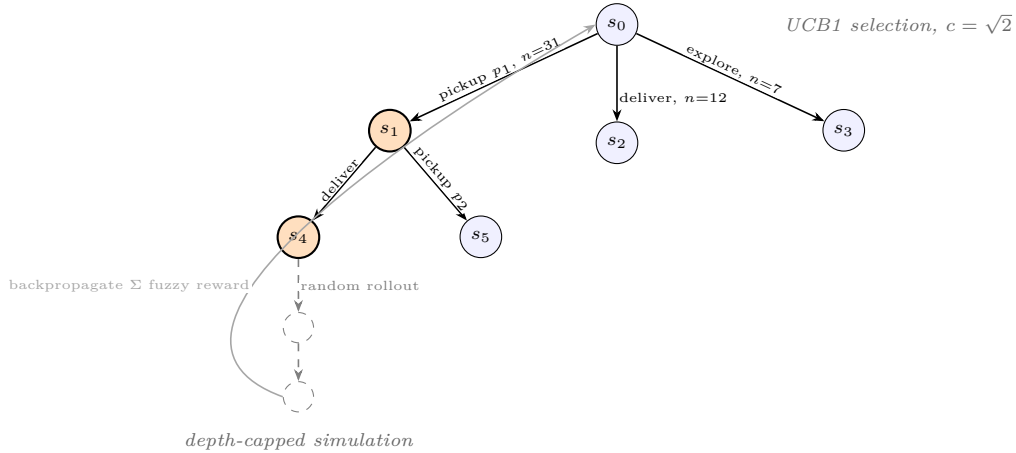


Figure 3: One MCTS iteration over the logical state space. Highlighted nodes form the most-visited (robust) path; the root child with the highest visit count (s_1 , pickup p_1) becomes the committed intention.

6 Fuzzy Logic as a Common Currency for Heterogeneous Metrics

Desire evaluations mix quantities that live on incompatible scales: traversal cost (grid steps), expected reward (game points), and urgency (an ordinal priority). Early ad hoc utility formulas (e.g. $\log(\text{reward})/\log(\text{cost} + 1)$ for pickups, $-\text{cost}$ for deliveries, pure staleness for exploration) were mutually incomparable, which is fatal for a search that must rank pickups *against* deliveries *against* exploration. The `FuzzyDesirabilityScorer` solves this with a Mamdani fuzzy inference system that maps every evaluation onto a single desirability scale $[0, 100]$ (Figure 4) — the same role fuzzy control plays in blending heterogeneous, uncertain signals in autonomous robot navigation [7] and in agent-based game behaviour design [8].

Fuzzification with game-derived scales. Cost and reward are fuzzified into LOW/MEDIUM/HIGH sets built from shoulder and triangular membership functions. Crucially, the set breakpoints are not constants but are derived per match from the live game configuration: cost breakpoints as fractions (0.15, 0.30, 0.50) of the Manhattan-style upper bound $width + height$ (empirically a much better ceiling for obstacle-laden grid traversal than the Euclidean diagonal), and reward breakpoints as multiples (0.3 $\times$, 1 $\times$, 1.75 $\times$) of the server-provided `reward_average`. Anchoring on the average alone (rather than $\mu \pm \sqrt{\sigma^2}$) keeps the scale meaningful when `reward_variance` is zero, and lets a multi-parcel delivery (summed reward) correctly read as HIGH. The scorer is instantiated once per search, so the scale is computed once from the belief and shared across all evaluations of that search.

Rule base and defuzzification. A compact rule base combines the memberships with min (fuzzy AND), e.g. *if cost is low and reward is high then desirability is high*; *if cost is high and reward is low then desirability is low*; urgency contributes two crisp rules that pull the output toward HIGH or LOW. Rules over *risk* were deliberately removed after calibration showed they fired uniformly (no desire currently populates a non-zero risk) and only biased every score downward; they will return once a real risk signal exists. Following Mamdani inference, each fired rule clips its output set at its firing strength, the clipped sets are aggregated by fuzzy union, and the crisp score is the centroid of the aggregate, sampled numerically over $[0, 100]$.

This score is used uniformly across the agent: as the per-step reward inside MCTS rollouts, and by the open-minded reconsideration policy when comparing a challenger desire against the current commitment — i.e. every comparison between options, wherever it happens, goes through the same fuzzy currency.

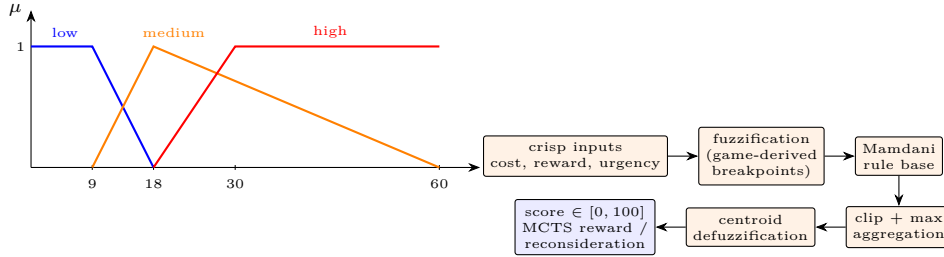


Figure 4: Left: cost membership functions on a 30 $\times$ 30 map (breakpoints at 9/18/30 derived from $width + height = 60$). Right: the Mamdani inference pipeline that turns heterogeneous metrics into a single comparable desirability score.

7 Planning, PDDL, and Anytime Plan Refinement

Once a desire is committed, `Planner` turns it into a concrete action plan by trying an ordered chain of pathfinders and taking the first that succeeds:

1. **A*** (`AStarPathFinder`) with Manhattan heuristic for plain navigation. Benchmarks (Section 9) show it comfortably fits inside a single movement tick on every official map, so it is always attempted first.
2. **PDDL** (`PDDL_PathFinder`) for situations A* cannot express — in particular maps where movable *crates* block the only corridors and must be pushed aside, a Sokoban-style problem that is genuinely relational rather than a shortest-path query — Sokoban is PSPACE-complete and a classic hard planning benchmark [9, 10], which makes it exactly the kind of sub-problem worth delegating to a planner.

7.1 PDDL domain and problem generation

The domain models the agent as a `sokoban` and crates as pushable objects, with typed `move--{up,down,left,right}` and `push--{up,down,left,right}` actions guarded by directional adjacency predicates, tile occupancy (`free-tile`), and `crates-allowed-tile` constraints, while a numeric fluent `total-distance` accumulates the plan metric to be minimised. The push actions are the relational heart of the domain:

```
(:action push-up
:parameters (?sokoban - sokoban ?crate - crate
             ?sokobanFromTile ?crateFromTile ?crateToTile - tile)
:precondition (and (on ?sokoban ?sokobanFromTile) (on ?crate ?crateFromTile)
                  (adjacent-up ?sokobanFromTile ?crateFromTile)
                  (adjacent-up ?crateFromTile ?crateToTile)
                  (free-tile ?crateToTile) (crates-allowed-tile ?crateToTile)))
```

```

:effect (and (free-tile ?sokobanFromTile) (not (on ?sokoban ?sokobanFromTile))
  (not (on ?crate ?crateFromTile)) (not (free-tile ?crateToTile))
  (on ?sokoban ?crateFromTile) (on ?crate ?crateToTile)
  (increase (total-distance) 1)))

```

Problems are generated on demand from the live belief by a dedicated generator supporting two occupancy modes: CLEAN-MAP (only static walls and crates constrain the problem) and FROZEN-SNAPSHOT (currently sensed rival positions are additionally frozen in as obstacles), trading optimism against congestion awareness. Tiles are emitted as objects `tile_x_y` only for walkable cells, with directional `adjacent-*` facts derived from the belief’s grid, so plan steps can be translated back to grid moves deterministically.

7.2 Anytime planning via an extended PaaS

Sokoban-like problems can take a satisficing planner well into seconds, but a satisficing *first* answer often arrives quickly and is then progressively improved. To exploit this, we extended the open-source planning infrastructure at both ends (an upstream pull request is in progress for the server side):

- **Client** — a published fork of `@unitn-asa/pddl-client` (`@matteoranzi/pddl-client`) whose `onlineSolver` gains (i) `onImprovedPlan(plan, metric)`, a callback invoked every time an anytime planner (e.g. OPTIC) reports a better solution while still searching, rather than a single result at the end; (ii) a per-request `maxTime` budget override; and (iii) an `AbortSignal` that not only stops the client from waiting but instructs the server to kill the planner process, freeing the worker.
- **Server** — a fork of AI-Planning `planning-as-a-service` (branch `feat/stream-anytime-progress`) that streams intermediate plans to the client instead of buffering until the solver exits.

The agent uses these primitives to bound planning latency: as soon as the first improved plan arrives, the solve is aborted and the plan executed, capping worst-case waiting at the configured budget while still benefiting from any early improvement:

```

const finalPlan = await onlineSolver(this.domainText, problem.toPddlString(), {
  maxTime: this.maxSolvingTimeS,
  signal: controller.signal,
  onImprovedPlan: (improvedPlan, metric) => {
    plan = improvedPlan; // best plan streamed by the anytime planner so far
    controller.abort(); // good enough: stop the solver, start executing
  },
});
plan ??= finalPlan;

```

The same machinery is the substrate for a planned refinement — hot-swapping the executing plan whenever the anytime planner later finds a strictly cheaper one, re-anchored to the agent’s current position (Section 10).

7.3 Plan execution and failure recovery

`PlanExecutor` executes plans one action at a time, re-checking before every step that the committed desire is still valid and still committed — so an intention switch or a belief change aborts the plan at the next step boundary, in classic BDI style. Failed actions (typically a rival occupying the next tile) are handled by composable recovery strategies implementing an escalation chain, e.g. *retry n times* → *replan from the current position (up to m times)* → *abort and re-deliberate*. The strategy object is created fresh per execution, keeping retry budgets per-plan, and the decorator-style composition (`ReplanThenAbort` wrapping `RetryThenAbort`) makes persistence policies declarative and swappable.

8 LLM-Controlled Agent

The LLM-controlled agent (Agent B) uses a local Large Language Model to transform natural-language requests into structured Deliveroo.js actions. It combines an autonomous fallback policy with an interactive chat interface: when no request is pending the agent behaves autonomously; when a user message arrives, the LLM temporarily takes responsibility for high-level planning. The LLM never receives unrestricted control: it returns a strict JSON object containing a textual response and a plan composed only of supported tools, conditions, and bounded loops, which the local application parses and executes.

8.1 Architecture and execution flow

The main component, `LlmAgent.js`, manages the `Deliveroo.js` connection, local world state (`WorldModel.js`: map, agent position, visible parcels, delivery and spawning tiles), autonomous behaviour, user requests, and LLM plan execution. When a chat request is received: the request is queued; the agent enters `LLM_MODE`, pausing autonomous decisions (avoiding the autonomous policy and the LLM plan steering the agent toward different targets); `promptBuilder.js` combines the user message, a compact world snapshot, and persistent memory (`memory.txt`, storing learned facts, reward observations, and strategy rules); `ollama.js` queries a local Ollama model; `parser.js` interprets and executes the returned plan; finally the agent resumes its fallback policy. A guard prevents concurrent request processing. Figure 5 summarises this pipeline.

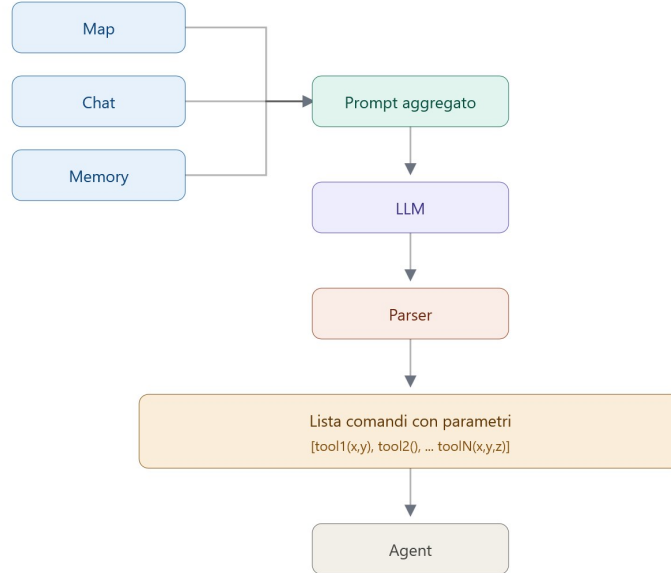


Figure 5: LLM-controlled agent pipeline. The map state, chat request, and persistent memory are aggregated into a single prompt; the LLM output is parsed into a parameterised command list ($\text{tool}_1(x,y), \dots, \text{tool}_N(x,y,z)$), which is then executed by the agent.

Outside `LLM_MODE`, the fallback strategy prioritises on-tile actions (pickup, delivery), then targets: deliver when carrying many parcels, move toward the nearest reachable free parcel, deliver when carrying with no visible parcel, otherwise explore. Navigation uses A* with visible agents treated as temporary obstacles, moving one step at a time so the world state refreshes between movements.

8.2 Constrained plan interpreter and tools

The LLM must return JSON only, with a `chat` field shown to the user and a `plan` field: a small domain-specific language of direct tool calls, `if/then/else` branches, and condition-guarded `loop` blocks — never arbitrary code. Tool names resolve through a predefined mapping, so the LLM can invoke only intentionally exposed functions: `move_to` (A* navigation with step limits), `pick_up_parcel` / `put_down_parcel`, `find_and_get_a_parcel` (a reactive search that refreshes its target after each movement, handling parcels that expire or are taken by rivals), `calculator`, `add_to_memory`, and `reply_to_server`. The interpreter applies defensive limits — bounded loop iterations, abort after repeated failed iterations, termination on fatal movement or network errors — because generated plans may be incomplete or based on outdated observations.

8.3 MCP-based control of a second agent

The system can coordinate a separate agent (`BdiLlmTestAgent.js`) through the Model Context Protocol: the second agent runs independently, connects to `Deliveroo.js`, and exposes selected capabilities via an HTTP MCP endpoint. The LLM agent drives it through explicit typed proxy tools (`agent_move_to`, `agent_pick_up_parcel`, `agent_put_down_parcel`, `agent_find_and_get_a_parcel`, `agent_get_state`), refreshing its state before evaluating conditions that involve it. This provides process separation: the LLM cannot touch the remote agent’s internals, only a restricted, typed action interface.

8.4 Web interface

The `webchat.js` component provides a browser chat panel with plan visualisation, receiving messages over HTTP and streaming replies via Server-Sent Events. Figure 6 shows a request mixing an informational question with a movement task; the LLM answers the question and emits four explicit `move_to` actions. More complex plans combine bounded local collection loops, delivery actions, and remote MCP commands for the second agent.

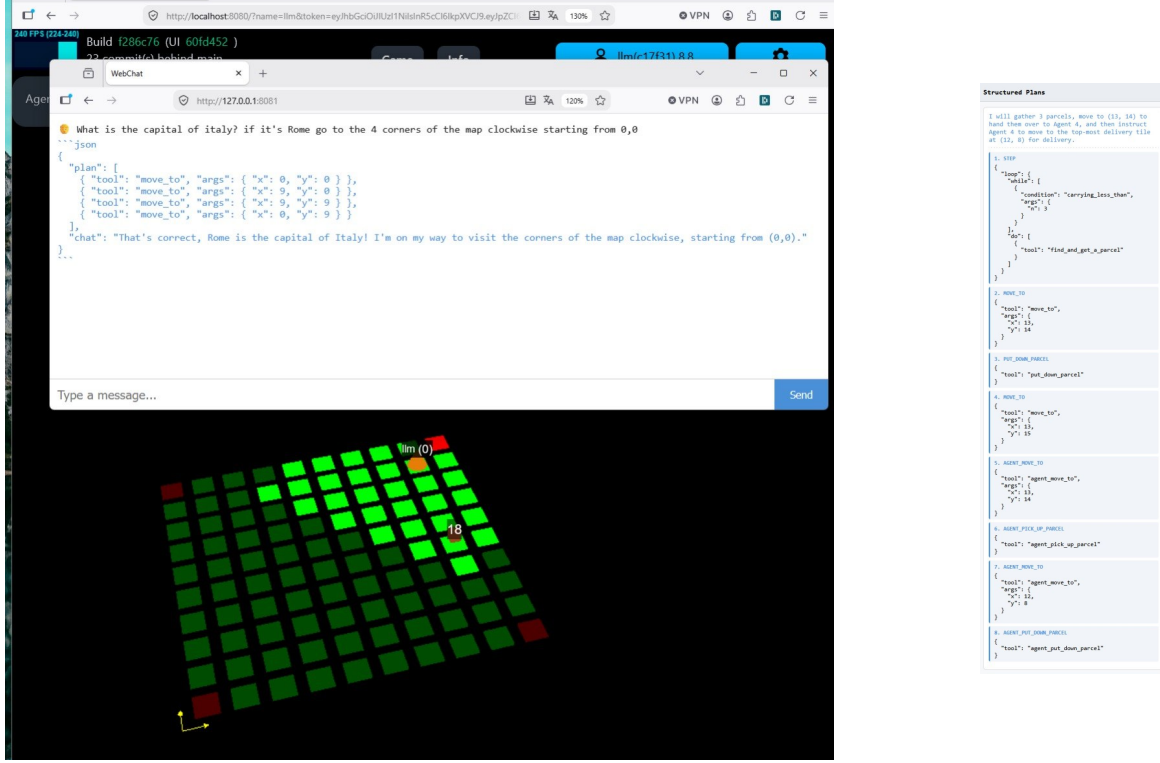


Figure 6: Left: web-chat example — the agent answers a natural-language request and generates a JSON plan with four coordinate-based movement actions. Right: a multi-step plan combining a bounded local collection loop, delivery actions, and remote MCP commands that direct the second agent to collect and deliver a hand-over parcel.

9 Validation and Calibration

Pathfinding benchmarks. A* was benchmarked on the official challenge maps by computing, from a random start, the path to every valid tile, averaged over 300 iterations per map (MacBook Pro M1 Max):

Map	Time (ms)	Valid tiles
empty_30	21.45	900
circuit_directional	16.20	485
hallways_interconnected	8.56	465
caothic_maze	7.16	432
wide_path	12.14	526
two_obstacles	11.36	524
crates_one_way	0.31	47

Table 1: A* pathfinding: total time to reach every valid tile from a random start. With move durations of at least 50 ms, planning always fits well within a single movement tick.

Data-driven fuzzy/MCTS calibration. Rather than hand-guessing constants, the live agent was instrumented: a one-time dump of the server-provided game configuration, a periodic [FUZZY-CALIBRATE] log recording observed min/max cost, reward, and score per desire category against the active fuzzy scale, and a per-search [MCTS-CALIBRATE] log recording root branching factor, total visits, and the top visited children.

This surfaced and fixed several concrete issues: cost breakpoints moved from the Euclidean diagonal (real A* costs reached 58 on a 30×30 map, past $\sqrt{w^2 + h^2} \approx 42$) to $w + h$; reward breakpoints were made robust to zero-variance reward distributions; delivery and exploration desires that silently reported a zero reward signal were repaired; and dead risk rules were removed. On the MCTS side, calibration revealed that the effective bottleneck was not the UCB1 constant but the root branching factor (57–95 children, dominated by per-spawner exploration desires): with 100 iterations, UCB1’s forced first visit to every child consumed most of the budget, and visit concentration improved sharply whenever branching dropped. This motivates the exploration-desire aggregation discussed in Section 10.

Scenario testing and observability. The BDI agent was validated on the Challenge 1 scenario maps (including the crate-blocked `crates_one_way` map exercising the PDDL pathfinder), and the LLM agent on Challenge 2 atomic-request scenarios through the chat interface. A live terminal UI (Figure 7) renders the full deliberation state — belief grid, current desire set with evaluations, the change-detection strategies that triggered the last cycle, and the committed intention with its fuzzy score — which proved essential both for debugging emergent behaviour and for the calibration campaign above.

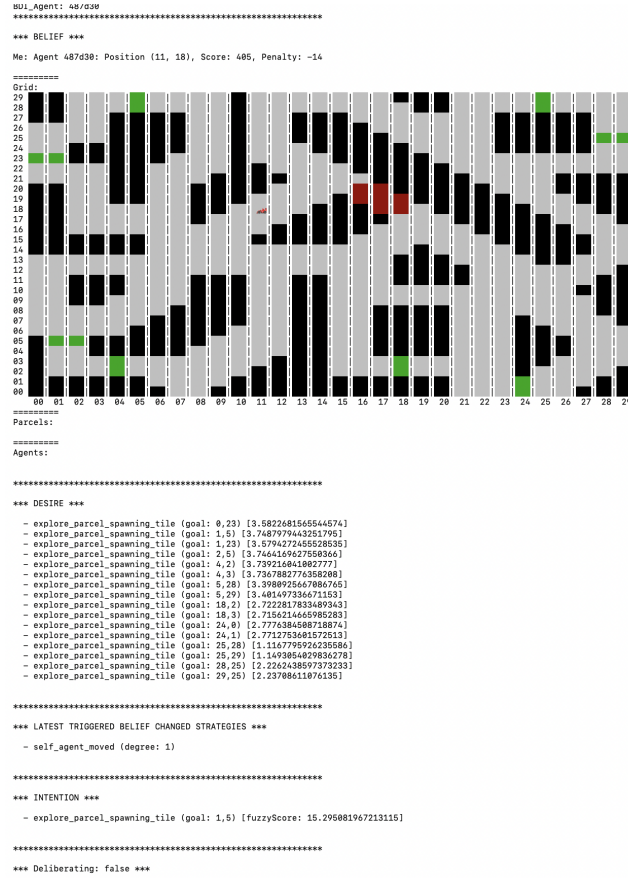


Figure 7: Live terminal UI of the BDI agent on a 30×30 map: belief grid (agent, walls, spawner and delivery tiles), the generated exploration desires with their staleness evaluations, the change-detection strategies fired on the last belief update, and the committed intention annotated with its fuzzy desirability score.

10 Conclusion and Future Development

The project delivers a BDI agent whose deliberation is genuinely modular — event-driven belief revision, desire-level MCTS lookahead over a lazily built logical state space, fuzzy inference as a common scoring currency, and a layered A*/anytime-PDDL planning stack with composable failure recovery — alongside an LLM-controlled agent with a safety-constrained plan interpreter and MCP-based control of a second agent. The interfaces introduced everywhere (selection strategies, reconsideration policies, pathfinders, recovery strategies, change-detection strategies) were a deliberate investment: each of the extensions below slots into an existing seam without touching the deliberation loop.

10.1 Future development

- **Hot-swappable intention strategies with online-tuned parameters.** Start each match with `GreedyIntentionStrategy` — cheap, reactive, and insensitive to branching factor — as a warm-up phase whose real purpose is data collection: observed cost and reward ranges per desire category, effective branching factors, and map-specific traversal statistics. Once enough evidence accumulates, a wrapper strategy hot-swaps to `MCTSIntentionStrategy` (and to a matching commitment policy) whose fuzzy breakpoints and search budget are calibrated from the collected statistics instead of configuration heuristics — automating online the manual run-log-tune loop used in Section 9. Because both strategies implement `IIntentionStrategy`, the swap is invisible to `Intention`.
- **Anytime plan hot-swapping during execution.** Extend the anytime PDDL integration so that improved plans arriving *while a plan is executing* are adapted to the agent’s current position and swapped in when strictly cheaper, rather than only bounding the initial solve.
- **Smarter exploration desires.** Replace per-spawner exploration desires with a small set of *observation hotspots* computed by a greedy maximum-coverage algorithm over the spawner set, simultaneously improving exploration quality and collapsing the MCTS root branching factor identified as the main search bottleneck.
- **Inter-agent coordination.** Implement message-based coordination between the BDI and LLM agents over the Deliveroo.js communication API — handshake and role assignment, periodic belief sharing (parcels and rivals seen outside the teammate’s sensing range), and parcel reservation to avoid targeting the same parcel; even minimal local communication is known to yield strongly coordinated multi-agent navigation [11]. In the same step, route in-game messages from the mission agent into the LLM agent’s request queue, so that mission-agent requests are handled through the same constrained planning pipeline as chat requests.
- **Automated comparative benchmarks.** Extend the benchmark harness beyond pathfinding to full-match comparisons (Greedy vs. MCTS intention selection, A*-only vs. A*+PDDL) with automated runs and result export across the challenge maps.
- **Risk as a live signal.** Populate the currently unused risk input of the fuzzy scorer from rival trajectory predictions (the belief base already tracks position histories), re-enabling the risk rule family removed during calibration.
- **Structured LLM memory.** Replace the plain-text persistent memory with a validated, structured store.

References

- [1] M. E. Bratman. *Intention, Plans, and Practical Reason*. Harvard University Press, 1987.
- [2] M. Świechowski, K. Godlewski, B. Sawicki, and J. Mańdziuk. Monte Carlo Tree Search: a review of recent modifications and applications. *Artificial Intelligence Review*, 56:2497–2562, 2023.
- [3] J. Norman and F. Rivest. Monte Carlo Tree Search satellite scheduling under cloud cover uncertainty. *arXiv:2405.20951*, 2024.
- [4] L. Liu, S. Chen, H. Jin, X. Deng, Y. Liu, and Y. Lin. Optimizing on-demand food delivery with BDI-based multi-agent systems and Monte Carlo tree search scheduling. *Scientific Reports*, 15:25083, 2025.
- [5] P. Caillou, B. Gaudou, A. Grignard, C. Q. Truong, and P. Taillandier. A simple-to-use BDI architecture for agent-based modeling and simulation. In *Proc. of the 11th Conference of the European Social Simulation Association (ESSA)*, 2015.
- [6] A. Traldi, F. Bruschetti, M. Robol, D. Calvaresi, M. Roveri, and P. Giorgini. Real-time BDI agents: a model and its implementation. *arXiv:2205.00979*, 2022.
- [7] A. Saffiotti. The uses of fuzzy logic in autonomous robot navigation. *Soft Computing*, 1(4):180–197, 1997.
- [8] Y. Li, P. Musilek, and L. Wyard-Scott. Fuzzy logic in agent-based game design. In *Proc. of the Annual Meeting of the North American Fuzzy Information Processing Society (NAFIPS)*, 2004.
- [9] T. Virkkala. *Solving Sokoban*. Master’s thesis, University of Helsinki, 2011.
- [10] N. S. Peters. *Solving Sokoban Efficiently: Search Tree Pruning Techniques and Other Enhancements*. Bachelor’s thesis, Technische Universität Berlin, 2023.
- [11] D. Hildreth and S. J. Guy. Coordinating multi-agent navigation by learning communication. *Proc. ACM on Computer Graphics and Interactive Techniques*, 2(2), 2019.

Appendix: PDDL pathfinding problem file

A complete (small) instance produced by the problem generator of Section 7, corresponding to a 5×4 map fragment in which a crate blocks the only corridor to the goal; repetitive tile and adjacency declarations are elided with [...].

```
;; problem file: deliveroo-pathfinding (generated from the live Belief)
(define (problem deliveroo-pathfinding)
  (:domain default)
  (:objects
    player - sokoban
    crate1 - crate
    ; tiles named tile_<x>_<y>; only walkable cells become objects
    tile_0_0 tile_1_0 tile_3_0
    tile_0_1 tile_2_1 tile_3_1 tile_4_1
    tile_0_2 tile_1_2 tile_3_2 tile_4_2
    tile_0_3 tile_1_3 tile_2_3 tile_3_3 - tile
  )
  ; =====
  ; MAP (bottom-left origin, x rightward, y upward)
  ;   3   f   f   f   f   .
  ;   2   f   f   .   C|ac f
  ;   1   f   .   ac   ac   f
  ;   0   A   f   .   0   .
  ; y/x  0   1   2   3   4
  ; f: free-tile          ac: crates-allowed-tile
  ; C: crate (tile_3_2)   A: agent start   0: goal
  ; .: hole (no tile object, no adjacency)
  ; =====
  (:init
    (on player tile_0_0)
    (on crate1 tile_3_2)
    ; crates-allowed tiles (static)
    (crates-allowed-tile tile_3_2)
    (crates-allowed-tile tile_2_1)
    (crates-allowed-tile tile_3_1)
    ; free tiles (walkable and currently unoccupied)
    (free-tile tile_1_0)
    (free-tile tile_3_0)
    (free-tile tile_0_1)
    [...]
    (free-tile tile_3_3)
    ; directional adjacency between walkable tiles only
    (adjacent-up tile_0_0 tile_0_1)
    (adjacent-down tile_0_1 tile_0_0)
    (adjacent-up tile_3_1 tile_3_2)
    (adjacent-down tile_3_2 tile_3_1)
    [...]
    (adjacent-right tile_2_3 tile_3_3)
    (adjacent-left tile_3_3 tile_2_3)
  )
  (:goal (on player tile_3_0))
  (:metric minimize (total-distance))
)
```

The only plan reaching `tile_3_0` must walk around via row 3, **push-down** the crate from `tile_3_2` to `tile_3_1`, come around through column 4, and **push-left** it aside onto the `crates-allowed` tile `tile_2_1` before finally descending the freed corridor — exactly the class of paths the A* pathfinder cannot produce.